

I'm re-running this as a follow-up and will refresh the report artifact under the requested `output_root` without redoing the full source audit. Then I'll return the required summary sections from the saved report.I'm writing the refreshed Markdown artifact now, keeping the same measured findings but packaging it in the required run folder and file header.

OVERALL CODEBASE RATING — FRONTEND MODERNIZATION

High Risk

The largest driver is massive component scale, with `App.jsx` , `Dashboard.jsx` , `StepFlowSelection.jsx` , and `StepIdeConfig.jsx` each carrying too much routing, orchestration, and form logic in one place.

EXECUTIVE SUMMARY

This workspace has a mature React frontend, and the dominant idiom is already function components with hooks on top of React 19.2.5. The main modernization gap is not framework migration; it is component scale and orchestration complexity, especially in `App.jsx` , `Dashboard.jsx` , and the setup flow screens. Legacy class-based UI is effectively absent aside from a single error boundary, so the codebase is mostly aligned with current React patterns. The most material risks are oversized components, repeated imperative state synchronization, and deep feature components that mix routing, auth, layout, and data orchestration. Overall, the codebase is usable and mostly modern, but the largest screens would benefit from extracting shared shell, state, and form primitives.

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	UI Component Duplication	Duplicate components %	<5%	5–10%	>10%	Not observed	GOOD
H2	Legacy Class-Based Components	Modern component adoption %	>90%	70–90%	<70%	99% modern (1 class boundary in 150 files)	GOOD
H3	Massive Components	Largest component LOC	<200	200–500	>500	4,172 LOC	HIGH RISK
H4	Global State Dependencies	Components reading global state %	<30%	30–60%	>60%	17/150 files (11.3%)	GOOD
H5	Complex State Management	Max prop-drilling depth	<3	3–5	>5	3 levels	MODERATE

Hotspot	Action	Rating	Priority
Massive Components	Split <code>App.jsx</code> , <code>Dashboard.jsx</code> , <code>StepFlowSelection.jsx</code> , and <code>StepIdeConfig.jsx</code> into shell, controller, and presentational pieces; extract navigation/auth logic and setup async flows into hooks.	HIGH RISK	CRITICAL
Complex State Management	Replace broad prop bundles with smaller domain hooks and localized feature state around <code>WorkflowStageCanvas</code> and the setup wizard.	MODERATE	MEDIUM

- Shared UI primitives will reduce screen-specific markup drift and make the setup and dashboard flows feel consistent.

- Smaller hooks and feature shells will make the hardest screens easier to test and safer to extend.
- Localized state and narrower prop surfaces will reduce accidental coupling between routing, auth, workflow execution, and configuration forms.
- The codebase will stay aligned with React 19 hooks-first conventions while keeping the existing central store for truly shared concerns.